

MODULE 10 · PART 3 - CTES AND RECURSION

CTEs and Materialisation

A piece of folklore that was true, stopped being true in 2019, and is still being taught.

PART 3 of 8 MODULE 10 of 26 RUNS about 32 min

Why this opens Part 3

- ◆ **Part 2 was about one clause. Part 3 is about query structure**, and structure is where stale advice does the most damage.
- ◆ **Nobody reasons about structure from first principles.** They repeat what they were told, because it worked once.
- ◆ **So this module is a worked example of checking a belief**, as much as it is about common table expressions.
- ◆ **The technique generalises.** Which engine, which version, and what does the plan actually say.

THE TAKEAWAY

Three questions settle almost every performance argument in SQL. Which engine, which version, and what does the plan say. This module is those three applied once.

What a CTE is, formally

- ◆ **A named subquery, scoped to one statement.** That is the entire definition, and everything else is implementation.
- ◆ **Several can be declared in one WITH clause**, comma-separated, and each can reference the ones declared before it.
- ◆ **None of them can reference a later one**, which is why a stack reads downward and cannot loop back.
- ◆ **Except a recursive one, which can reference itself.** That single exception is the whole of module eleven.

THE TAKEAWAY

A name, a scope, and forward-only references. From those three properties everything in this module and the next one follows.

Reading a WITH clause

WRITTEN	SAID
WITH a AS (...)	let a be the result of this query
WITH a AS (...), b AS (SELECT ... FROM a)	and let b be built from a
WITH a AS MATERIALIZED (...)	compute a once, in full, before anything else
WITH a AS NOT MATERIALIZED (...)	substitute a's text wherever it is used
WITH RECURSIVE a AS (...)	let a be defined partly in terms of itself

THE TAKEAWAY

Row four is the accurate reading of inlining. The block's text is substituted at each reference, which is why an inlined block referenced twice runs twice.

"CTEs are slow, because they are an
optimisation fence."

What the fence actually was

- ◆ **PostgreSQL always materialised a CTE in full**, computing the entire result before the outer query ran.
- ◆ **So the planner could not push a restriction into it.** A filter in the outer query could not reduce the work inside.
- ◆ **A CTE over a large table therefore read the whole table**, even when the outer query wanted three rows out of it.
- ◆ **That is the whole of it.** Not a slow implementation, and not overhead. A missed optimisation, with a specific shape.

THE TAKEAWAY

The fence was never about the CTE being expensive to build. It was about the outer query's filters arriving too late to help.

The demonstration

AS A CTE, BEFORE PG12

The filter arrives too late

```
1 WITH all_sales AS (  
2   SELECT * FROM fact_sales_line  
3 )  
4 SELECT * FROM all_sales  
5   WHERE store_key = 41;
```

Materialises 2.41 billion rows, then filters to one store's worth.

AS A SUBQUERY

The filter is pushed down

```
1 SELECT * FROM (  
2   SELECT * FROM fact_sales_line  
3 ) t  
4   WHERE store_key = 41;
```

The planner pushes store_key = 41 inside, and reads one store's worth.

THE TIEBREAKER

Two queries with identical meaning and wildly different cost, on PostgreSQL 11. That gap is the entire basis of the folklore.

And people used it deliberately

- ◆ **A fence is sometimes what you want.** It stops the planner making a choice you know to be bad.
- ◆ **So CTEs became PostgreSQL's unofficial query hint**, in a database that has always refused to add real ones.
- ◆ **Which conflated two things.** The meaning of a query, and the execution policy for it, expressed in the same syntax.
- ◆ **That conflation is why the change took so long.** Removing the fence would break queries that depended on it.

THE TAKEAWAY

PostgreSQL has never had optimizer hints, on principle. The CTE fence was the hint people had, and it worked by accident of implementation.

The same logic, four ways

```
1 -- 1. inline subquery
2 SELECT * FROM (SELECT ... ) t WHERE ...;
3
4 -- 2. common table expression
5 WITH t AS (SELECT ...) SELECT * FROM t WHERE ...;
6
7 -- 3. temporary table
8 CREATE TEMP TABLE t AS SELECT ...;
9 SELECT * FROM t WHERE ...;
10
11 -- 4. view
12 CREATE VIEW t AS SELECT ...;
13 SELECT * FROM t WHERE ...;
```

- 8 Only this one is two statements. That single difference decides more architecture questions than performance ever does.

THE TAKEAWAY

Four forms of one idea. Three are query constructs and one is a schema object, and the boundary between those two is where teams argue.

CTEs and the evaluation order

- ◆ **Each block is a complete query with its own ten steps.** Its own WHERE, its own GROUP BY, its own windows.
- ◆ **Which is why a block creates a scope where your aliases exist**, as module one showed. They are columns by the next level.
- ◆ **And why an ORDER BY inside a block is not a promise**, because a block produces a relation and a relation has no order.
- ◆ **So a stack is ten steps, then ten more, then ten more.** Not one big query with a longer list.

THE TAKEAWAY

Module one's list applies once per block, not once per statement. That is the mental model that makes a stacked query predictable.

A block is where a name becomes real

```
1 WITH priced AS (  
2     SELECT sale_key, store_key,  
3           net_amount - discount_amount AS net_after_disc  
4     FROM fact_sales_line  
5 )  
6 SELECT store_key, SUM(net_after_disc) AS total  
7     FROM priced  
8 WHERE net_after_disc > 0  
9 GROUP BY store_key;
```

8 The outer WHERE filters on a name created in the inner SELECT. Impossible in one block, ordinary across two.

THE TAKEAWAY

This is why analytical SQL is stacked, and it has nothing to do with performance. A block is the only construct that turns an expression into a column.

PostgreSQL 12, released October 2019. The fence became **a choice**.

The rule, exactly

THE CTE IS

Non-recursive, side-effect-free, referenced once

Non-recursive, side-effect-free, referenced twice or more

Recursive

Data-modifying, so INSERT, UPDATE or DELETE

Anything, with MATERIALIZED

Anything eligible, with NOT MATERIALIZED

WHAT HAPPENS BY DEFAULT

Inlined. The fence is gone and restrictions push down.

Materialised, as before. Computed once, reused.

Always materialised. There is no choice here.

Always materialised, and always run to completion.

Materialised. The old behaviour, on request.

Inlined, even when referenced many times.

THE TAKEAWAY

Three conditions decide the default: recursive or not, side-effect-free or not, referenced once or more. Everything else is the two keywords.

The reasoning, in the committers' own terms

- ◆ **Materialising avoids computing an expensive CTE twice.** That is the argument for it, and it only applies if it is called twice.
- ◆ **Even then it can be a net loss,** if each call has restrictions that would let only a small part be computed.
- ◆ **So the default is a heuristic, not a rule.** Referenced once means inline, because there is nothing to reuse.
- ◆ **And MATERIALIZED exists for the deliberate fence,** which the commit names as the main reason somebody would want it.

THE TAKEAWAY

The second point is scenario two. Computing an expensive block once and reusing it is not automatically cheaper than computing a small part of it three times.

The two keywords

```
1 WITH expensive AS MATERIALIZED (  
2   SELECT store_key, SUM(net_amount) AS sales  
3   FROM fact_sales_line GROUP BY store_key  
4 )  
5 SELECT * FROM expensive WHERE sales > 100000;  
6  
7 WITH narrow AS NOT MATERIALIZED (  
8   SELECT * FROM fact_sales_line  
9 )  
10 SELECT * FROM narrow WHERE store_key = 41;
```

- 1 Force the fence. Compute the aggregate once, and do not let the outer filter change how it is built.
- 7 Forbid the fence. Push the store filter inside, even if the block is referenced several times.

THE TAKEAWAY

Two keywords, and both are decisions about execution rather than meaning. Neither changes what the query returns, on any data.

A detail worth knowing

- ◆ The **MATERIALIZED** keyword was backported to older PostgreSQL as a no-op, in October 2019, before 12 was widely deployed.
- ◆ So applications could add it in advance, run unchanged on 11, and keep the old behaviour when the database moved to 12.
- ◆ Without it, teams faced upgrading application and database simultaneously, or accepting a regression in between.
- ◆ **NOT MATERIALIZED** on a data-modifying CTE is silently ignored, which was debated in 2019 and left as it stands.

THE TAKEAWAY

A keyword deliberately accepted and ignored by older versions, purely so that people could prepare. That is a good measure of how disruptive the change was expected to be.

Kestrel's store-ops replica is upgraded from PostgreSQL 11 to 16. Statistics are rebuilt afterwards. One nightly reconciliation query goes from forty seconds to twenty-five minutes; everything else is unchanged or faster.

The query has one CTE that aggregates a large table, referenced exactly once, with a selective filter in the outer query. The output is byte-identical before and after.

What is the most likely cause?

- A The upgrade changed the default NULL ordering, so the ordering inside the CTE now differs.
- B PostgreSQL 12 onwards inlines a single-reference CTE, removing a fence this query relied on for its plan.
- C Table statistics are stale after a major-version upgrade, so the planner has no useful row estimates.
- D CTEs are slower than temp tables. Rewrite the block as a temporary table and the regression goes away.

◆ PAUSE AND DECIDE

What is the most likely cause?

A NULL ordering changed

It did not, and it would change the output rather than the runtime. The scenario says the output is byte-identical.

C Stale statistics

The right thing to check first, always, and the scenario rules it out by saying statistics were rebuilt afterwards.

B The fence was removed

Non-recursive, side-effect-free, referenced once. All three conditions for inlining are met, and the plan the query relied on is gone.

D Rewrite as a temp table

Would work, by materialising the block. It reaches the right outcome through folklore rather than diagnosis, and MATERIALIZED is one word.

THE SIGNAL

"Referenced exactly once." That phrase is the whole diagnosis. It is precisely the condition that flipped in PostgreSQL 12, and the query met all three.

The fence was **a PostgreSQL behaviour.**
Most engines never had it at all.

CTE handling, by engine

ENGINE	WHAT IT DOES WITH A NON-RECURSIVE CTE
PostgreSQL 12+	Inlines if referenced once, materialises if referenced more.
PostgreSQL 11 and earlier	Always materialises. The version the folklore describes.
SQL Server	Always an inline view. Referenced twice means computed twice.
Snowflake, Redshift, BigQuery	Inline views. Reuse of a repeated block is not promised.
Oracle	Decides by cost, and offers MATERIALIZE and INLINE hints.

THE TAKEAWAY

SQL Server is the opposite hazard. A CTE referenced three times is computed three times, and there is no keyword to stop it.

So the advice inverts by engine

- ◆ **On old PostgreSQL, a referenced-once CTE was the expensive case.** The fix was to write a subquery instead.
- ◆ **On SQL Server, a referenced-many-times CTE is the expensive case,** because each reference recomputes it.
- ◆ **On modern PostgreSQL, neither is a problem by default,** because the default already distinguishes those two cases.
- ◆ **Which is why the folklore is so persistent.** Everybody's experience was real, and none of it generalised.

THE TAKEAWAY

Two engines, two opposite failure modes, one piece of advice circulating for both. The advice was never about CTEs. It was about one planner.

Pushing a predicate down by hand

```
1 -- the planner cannot push into a fence
2 WITH heavy AS MATERIALIZED (
3   SELECT store_key, SUM(net_amount) AS sales
4   FROM fact_sales_line GROUP BY store_key)
5 SELECT * FROM heavy WHERE store_key = 41;
6
7 -- so push it yourself
8 WITH heavy AS MATERIALIZED (
9   SELECT store_key, SUM(net_amount) AS sales
10  FROM fact_sales_line
11   WHERE store_key = 41
12   GROUP BY store_key)
13 SELECT * FROM heavy;
```

- 11 The same predicate, written inside. The scan now reads one store, and the fence costs nothing.

THE TAKEAWAY

If you want a fence and an early filter, write the filter inside. The fence stops the planner moving it, not you.

Predicate pushdown, generally

- ◆ **Pushing a filter closer to the scan is the single largest optimisation** most queries get, and it is usually automatic.
- ◆ **Things that block it: a fence, a volatile function, an outer join, and sometimes a window function.**
- ◆ **A window function blocks it for a reason.** Filtering before the window would change the window's answer, which module three covered.
- ◆ **So a filter that cannot be pushed is sometimes correctness**, not a missed optimisation. Check which before forcing it.

THE TAKEAWAY

The planner refusing to push a predicate past a window is protecting your answer. Forcing it past by hand is a change of meaning, not a tuning step.

Materialising is not a workaround.
Sometimes it is **the correct instruction**.

Four reasons to force materialisation

- ◆ **Expensive and genuinely reused.** A heavy aggregate referenced four times, where each reference wants all of it.
- ◆ **The block must be evaluated exactly once.** A random sample, a sequence, a timestamp, anything not stable.
- ◆ **The planner is choosing badly** and you have measured it. A fence is a blunt hint and it is the hint PostgreSQL has.
- ◆ **You want a stable snapshot**, so that two references cannot disagree with each other about what the block contained.

THE TAKEAWAY

The second and fourth are correctness reasons, not performance ones. They are the only two on this slide that cannot be settled by measuring.

Two references, one block, two answers

```
1 WITH top_100 AS (  
2     SELECT customer_key, annual_spend  
3     FROM vw_member_annual_spend  
4     ORDER BY annual_spend DESC  
5     LIMIT 100  
6 )  
7 SELECT (SELECT SUM(annual_spend) FROM top_100) AS total,  
8        (SELECT COUNT(*) FROM top_100) AS n;
```

- 4 Not a total order. Many members share an annual spend, so the hundredth place is contested.
- 7 Two references. If the block is inlined, it is evaluated twice, and the ties may break differently each time.

THE TAKEAWAY

Module two, arriving inside a CTE. Materialisation makes the two references agree by accident. A total order makes them agree on purpose.

A Postgres 16 report has one CTE that scans and aggregates a 400 million row table. It is referenced three times, and each reference filters to a different region, covering about a seventh of the data each.

The query takes eleven minutes. A colleague suggests the CTE is being computed once and reused, which should be efficient, and that the problem must lie elsewhere.

What is the most effective change?

- ☐ A Add NOT MATERIALIZED, so each reference pushes its own region filter down into the block.
- ☐ B Restructure so the block is referenced once, and join the three regional slices together afterwards.
- ☐ C Materialise it into an indexed temporary table, so each reference can seek rather than scan.
- ☐ D Increase work_mem, so that the materialised intermediate stays in memory rather than spilling.

◆ PAUSE AND DECIDE

What is the most effective change?

A Add NOT MATERIALIZED

Three scans of a seventh each is under half the work of one scan of everything. The commit message names exactly this case.

C Indexed temp table

Helps, and pays a full scan plus a write plus an index build first. Worth it if several queries share the intermediate, and not for one.

B Restructure to one reference

Arrives at a similar plan through a rewrite. It is more work than one keyword and it changes code somebody has to review.

D More work_mem

Makes the materialisation cheaper rather than unnecessary. It optimises the step that should not be happening.

THE SIGNAL

"About a seventh of the data each." Three narrow scans totalling three sevenths beat one full scan. Reuse is only a saving when the references want the same rows.

Let it decide, or decide for it

TRUST THE PLANNER

No keyword

- Adapts as data changes
- Adapts as the engine improves
- Occasionally chooses badly
- Nothing to revisit later

FORCE IT

MATERIALIZED

- Stable, and you measured it
- Frozen against future improvement
- Outlives the problem it solved
- Nobody knows why it is there

THE TIEBREAKER

A forced decision is correct on the day it is made and never re-examined. Write the reason in a comment or it becomes folklore of its own.

A block that must run once

```
1 WITH sample AS MATERIALIZED (  
2   SELECT customer_key  
3     FROM dim_customer  
4     WHERE is_current  
5     ORDER BY RANDOM()  
6     LIMIT 5000  
7 )  
8 SELECT 'treatment' AS arm, COUNT(*) FROM sample  
9 UNION ALL  
10 SELECT 'control', COUNT(*) FROM vw_eligible  
11 WHERE customer_key NOT IN (SELECT customer_key FROM sample);
```

- 1 MATERIALIZED here is a correctness requirement. Two evaluations of RANDOM would produce two different samples.
- 11 The second reference must see the same 5,000 customers, or the two arms overlap and the test is void.

THE TAKEAWAY

Not a tuning decision. If this block is evaluated twice, the treatment and control groups share members and nobody finds out until the results are analysed.

Sampling and reproducibility

- ◆ **ORDER BY RANDOM()** is not reproducible between runs, which is usually fine and occasionally catastrophic.
- ◆ **Hash the key instead for a stable sample.** A modulus on a hash of the customer key gives the same set every time.
- ◆ **Which removes the materialisation requirement entirely**, because a stable expression gives the same answer however often it runs.
- ◆ **That is the better fix.** Make the block deterministic rather than making the engine promise to run it once.

THE TAKEAWAY

The same pattern as module two's ordering. A correctness requirement met by an execution policy is fragile. Met by determinism, it is not.

A CTE is not the only way to name an intermediate. There are **four**, and they differ in what survives.

Four ways to name an intermediate

FORM	WHAT YOU GET
Inline subquery	Nothing named. One statement, optimised as a whole.
CTE	A name, one statement, and a materialisation decision.
Temporary table	Real rows, real statistics, an index if you build one. Survives the session.
View	A saved query, re-run every time. No rows and no statistics.
Materialised view or table	Stored rows, shared across sessions and users, and stale until refreshed.

THE TAKEAWAY

Only the last two are visible to anybody else. That is usually the decision, and it is a modelling one rather than a query one.

CTE, or temporary table

CTE

Inside one statement

- No write, no cleanup
- Estimated rows, not counted
- Cannot be indexed
- Gone when the statement ends

TEMP TABLE

Across statements

- Costs a write and a drop
- Real statistics after ANALYZE
- Can carry an index
- Reusable by later statements

THE TIEBREAKER

The statistics line is the one that decides hard cases. A planner guessing at a CTE's row count will sometimes guess very badly, and a temp table removes the guess.

When the temp table wins

- ◆ **Several separate statements need the same intermediate.** A CTE cannot cross a statement boundary at all.
- ◆ **The row estimate is badly wrong** and the plan below it is suffering. Real statistics fix it outright.
- ◆ **You need an index on the intermediate**, usually because something joins to it many times.
- ◆ **And it loses whenever the write costs more than the saving**, which on a small intermediate is most of the time.

THE TAKEAWAY

The first reason is the one that actually decides it in practice. A pipeline made of several statements needs something that survives them, and that is not a CTE.

A Redshift transformation is written as a chain of fourteen stacked CTEs. One block in the middle aggregates the sales fact and is referenced by five later blocks. The job takes ninety minutes.

An engineer proposes adding MATERIALIZED to the shared block so it is computed once. The team is on Redshift. A second engineer suggests reducing the number of CTEs instead.

What should be done?

- ☐ A Reduce the chain to fewer CTEs, since deeply nested blocks are what make the optimiser struggle.
- ☐ B Add MATERIALIZED to the shared block, which is the documented way to compute it once.
- ☐ C Increase the cluster size, since the job is scanning a large fact table five times over.
- ☐ D Persist the shared block as a temporary table and read it from each of the five downstream blocks.

◆ PAUSE AND DECIDE

What should be done?

A Fewer CTEs

Nesting depth is a readability property rather than a cost one. Collapsing fourteen blocks into four would not change how often the shared one runs.

C Bigger cluster

Makes five scans faster rather than making them one. It pays for the repetition instead of removing it.

B Add MATERIALIZED

The keyword is PostgreSQL-specific. Redshift has no such option, and the statement would simply fail to parse.

D A temporary table

Five references become one write and five cheap reads, and the intermediate gains real statistics for the blocks below it.

THE SIGNAL

"The team is on Redshift." One clause rules out the option that sounds most informed. A keyword that solves this problem exists, and not on this engine.

A view is a CTE somebody saved

- ◆ **A view is a stored query, expanded into whatever selects from it.** Much like an inlined block, and it has a name in the schema.
- ◆ **Which makes it the right home for logic several queries share,** because a block cannot be shared between statements at all.
- ◆ **It has no rows and no statistics,** so it does not solve an estimation problem. It solves a duplication problem.
- ◆ **A materialised view does have rows,** and is therefore stale between refreshes, which is a modelling decision rather than a query one.

THE TAKEAWAY

Copying a CTE into three queries is the moment it should have been a view. Copying a view into three schemas is the moment it should have been a model.

The same block, promoted

```
1 -- in one query
2 WITH baskets AS (
3     SELECT txn_id, store_key, SUM(net_amount) AS basket_value
4     FROM fact_sales_line GROUP BY txn_id, store_key
5 )
6 SELECT ... FROM baskets ...;
7
8 -- shared by many
9 CREATE VIEW vw_basket AS
10    SELECT txn_id, store_key, SUM(net_amount) AS basket_value
11    FROM fact_sales_line GROUP BY txn_id, store_key;
```

9 Identical SQL, promoted to a schema object. Every module in this course that said vw_basket meant this.

THE TAKEAWAY

The promotion costs nothing technically and changes everything organisationally. It now has an owner, a permission model, and consumers who will notice if it changes.

What a temp table actually costs

- ◆ **A write, which on a distributed engine may also be a redistribution.** The rows have to land somewhere before they can be read.
- ◆ **An ANALYZE, if you want the statistics that were the point of creating it.** Creating it alone does not give you them everywhere.
- ◆ **Cleanup, and a session that now has state.** Two runs in one session will collide unless the table is dropped or replaced.
- ◆ **So it wins on reuse and on estimation, and loses on everything small.** Under a few million rows it is rarely worth it.

THE TAKEAWAY

The statistics are the reason to do it and they are not automatic. A temp table created and never analysed gives the planner a default guess, which is what you were escaping.

The performance question is settled. The **readability** argument was always the real one.

A stack that reads like a paragraph

```
1 WITH baskets AS (           -- one row per receipt
2   SELECT txn_id, store_key, SUM(net_amount) AS basket_value
3   FROM fact_sales_line GROUP BY txn_id, store_key
4 ), store_day AS (           -- one row per store per day
5   SELECT store_key, COUNT(*) AS baskets,
6         AVG(basket_value) AS mean_basket
7   FROM baskets GROUP BY store_key
8 ), ranked AS (             -- position within the region
9   SELECT s.region_key, d.*,
10         RANK() OVER (PARTITION BY s.region_key
11                     ORDER BY d.mean_basket DESC) AS rnk
12   FROM store_day d JOIN dim_store s USING (store_key)
13 )
14 SELECT * FROM ranked WHERE rnk <= 3;
```

- 1 Each block is named for its grain, and the comment states that grain in four words.

THE TAKEAWAY

Three named grains and a filter. Somebody who has never seen this query can tell you what each level holds without reading any of the SQL.

Naming a block

- ◆ **Name it for what one row represents**, not for what the step does. 'baskets', not 'group_sales_by_txn'.
- ◆ **Comment the grain in four words**. One row per receipt. One row per store per day. That is the whole comment.
- ◆ **Avoid numbered names**. step1, step2 and step3 tell a reader nothing and survive every rename of what they contain.
- ◆ **A block whose grain you cannot state in one line** is usually doing two things and should be two blocks.

THE TAKEAWAY

The last point is a genuine test. If naming the grain is hard, the difficulty is in the query rather than in the naming.

The debugging property

- ◆ **Comment out the final SELECT and select from any block.** The query still parses and you see that level's output.
- ◆ **Which makes row counts checkable at every grain,** so the level that changed the count is found in one pass.
- ◆ **An inline subquery cannot do this,** because there is no name to select from and the structure has to be taken apart.
- ◆ **This is the property people mean by readable,** and it is really about inspectability rather than appearance.

THE TAKEAWAY

The most common defect in a long query is a grain change nobody noticed. Being able to count rows at each level turns that from a hunt into a bisection.

How many levels is too many

- ◆ **There is no planner limit worth worrying about.** Fourteen blocks is not a performance problem on any modern engine.
- ◆ **There is a human limit, and it is about references rather than depth.** A block used by five others is a shared dependency.
- ◆ **Once a block has several consumers, it wants to be a model,** with a name, tests and an owner. Not a block inside one query.
- ◆ **So the question is not how deep.** It is how many things depend on this level, and whether they should.

THE TAKEAWAY

Scenario three's fourteen-block chain was not too deep. It had one block with five consumers, which is a modelling signal rather than a query one.

What materialisation changes, and what it does not

PROPERTY	DOES MATERIALISATION CHANGE IT?
The rows returned	No, for any deterministic block. That is the guarantee.
... for a non-deterministic block	Yes. Two evaluations can differ. Scenario four.
The plan	Yes. That is the purpose of the keyword.
The runtime	Sometimes better, sometimes worse. Measure it.
Row estimates below the block	Yes. Materialised and inlined estimate differently.

THE TAKEAWAY

Row two is the one that matters. Materialisation is sold as a performance control and can silently become a correctness control.

Reading the plan

- ◆ **Look for a CTE Scan node.** Its presence means the block was materialised and the fence is there.
- ◆ **Its absence means the block was inlined,** and you will see the underlying scan with your outer filter attached to it.
- ◆ **Compare estimated rows with actual rows,** using EXPLAIN ANALYZE. A large gap is where the plan went wrong.
- ◆ **Both keywords are visible in the plan,** so you can confirm the change rather than assuming it worked.

THE TAKEAWAY

This is a ten-second check and it settles every argument in this module. The plan says whether the fence is there, and no amount of folklore outvotes it.

CTEs in a modelling tool

- ◆ **A transformation tool turns each model into a table or a view**, so a model boundary is a materialisation boundary.
- ◆ **Which means the decision this module is about gets made per model**, in configuration, rather than per query in SQL.
- ◆ **And a model referenced by five others is materialised by default**, which is scenario three solved structurally.
- ◆ **Blocks inside a model are still blocks**. Everything in this module applies within the model's own SQL.

THE TAKEAWAY

The modelling layer answers the reuse question and does not answer the within-query one. Both decisions still exist, at two different scales.

One block, one assertion

```
1 WITH baskets AS (  
2     SELECT txn_id, store_key, SUM(net_amount) AS basket_value  
3     FROM fact_sales_line GROUP BY txn_id, store_key  
4 )  
5 -- assertion: one row per txn_id, and nothing lost  
6 SELECT COUNT(*)                AS rows_out,  
7        COUNT(DISTINCT txn_id)   AS distinct_txns,  
8        SUM(basket_value)        AS total_value  
9 FROM baskets;
```

7 These two must be equal, or the block's declared grain is not its actual grain.

THE TAKEAWAY

Run this once per block while building the stack. The level where rows_out stops equalling distinct_txns is the level that broke the grain.

A Postgres 16 query defines a CTE selecting the top 100 products by units sold, ordered by units descending with no tiebreaker. Two later blocks reference it, one summing revenue and one counting categories.

An engineer adds NOT MATERIALIZED after reading that inlining is faster. The query speeds up. A week later, the revenue total and the category count start describing slightly different sets of products.

What happened?

- ☐ A The LIMIT should be applied inside each reference rather than in the shared block.
- ☐ B NOT MATERIALIZED is not permitted on a block containing LIMIT, and is being silently ignored.
- ☐ C Inlining evaluates the block twice, and a non-unique ORDER BY lets each evaluation return a different 100 rows.
- ☐ D The two references should be combined into one, so that only a single evaluation is possible.

◆ PAUSE AND DECIDE

What happened?

A Move the LIMIT inside

Puts the same ambiguous ordering in two places instead of one. Both references still pick their own hundred, and now it looks deliberate.

C Two evaluations, ambiguous order

Materialisation was hiding an incomplete ordering. Removing it exposed a defect that was always in the query.

B Silently ignored

DML blocks are the case where the keyword is ignored. A LIMIT is not, and the query did get faster, which shows inlining happened.

D Combine the references

Would work and treats the symptom. The ordering is still ambiguous, and the next person to reference that block meets it again.

THE SIGNAL

"The query speeds up" and then diverges. Materialisation was not making the query correct. It was evaluating an ambiguous block once, so nobody could see the ambiguity.

Deciding, in four questions

01

Engine

Which engine? The default behaviour and the available keywords both depend on it.

02

References

How many times is the block used, and does each use want all of it or a slice?

03

Stability

Must it evaluate exactly once? If so, that is a correctness requirement, not a tuning one.

04

Lifetime

Does anything outside this statement need it? If so, it is a table, not a block.

THE TAKEAWAY

Only question two is about speed. The other three are about portability, correctness and scope, and they have answers that measuring cannot give you.

Testing a stacked query

- ◆ **Record the row count at every level**, once, while it works. Those numbers are the specification of the stack.
- ◆ **Assert the grain at every level**, with the count against the distinct key. Two numbers per block.
- ◆ **Assert the measure total is preserved** across levels where it should be, which catches a join that fanned out.
- ◆ **Testing only the final output tells you something broke**, and not where. A stack gives you the where for free.

THE TAKEAWAY

Three assertions per block, and the cost of them is that somebody has to state each block's grain out loud. That statement is worth more than the tests.

A block is **a claim about grain**.
Materialisation is a claim about execution.
Never let the second decide the first.

The confusions, settled

CLAIM	THE TRUTH
CTEs are slow	Was true on PostgreSQL 11 and earlier. Not since 2019.
CTEs are always materialised	Only if recursive, data-modifying, or referenced more than once.
Materialising is always faster when reused	Not when each reference wants a different slice.
A subquery is faster than a CTE	Identical plans on every engine in current use.
MATERIALIZED is standard SQL	PostgreSQL only. Oracle has hints; nobody else has either.
Deep nesting slows the planner	Not measurably. Reference count matters, depth does not.

THE TAKEAWAY

Six claims, and every one of them was true somewhere at some point. That is what makes stale performance advice so durable.

Five habits

- ◆ **Stack CTEs, one grain per level, and comment the grain.** The readability argument was always the real one.
- ◆ **Check the plan before believing anything about materialisation,** including anything said in this module.
- ◆ **Reach for a temp table when a block has several consumers,** or when it has to survive a statement boundary.
- ◆ **Never let correctness depend on a materialisation decision.** Make the ordering total instead.
- ◆ **Date the performance advice you are repeating.** If you cannot say which engine and which version, do not repeat it.

THE TAKEAWAY

The fourth is the only one that prevents a wrong answer. The fifth is the one that would have prevented this entire module from being necessary.

One kind of CTE is **always** materialised, and it is the only one that can refer to itself.